CNT4713 Chat Project

Overview

In this project, you will design and implement a TCP-based client-server chat application using **only socket programming** in the **Python** programming language. The server must support multiple concurrent clients and allow message broadcasting and private messaging. There are two parts to this project, 1) the **server**, and 2) the **client**.

Constraints

- Use socket programming only.
- Use TCP sockets.
- Programming language: Python
- No external messaging frameworks.
- Client and server must run as separate processes.
- Server file **must be named**: server.py
- Client file **must be named**: client.py
- You must explicitly follow the input and output guidelines.

An autograder will grade the code submissions. This autograder executes pre-written versions of the client and server separately with your submission (your client + known server, your server + known client). You must therefore explicitly name the files as instructed. Also, you must explicitly follow the commands, flow, input, and output given. **From a runtime perspective, the usernames, ports, IP addresses, and messages are all variable**. The grader will user their own values for these settings and the expected out **must** match. All submissions which do not match will received a **zero**.

The Server

The following table outlines the commands the server will accept. All commands will return status code **200** when the server executes the command successfully, and **500** when the command fails.

Command	**Example**
connect <ip> <port>	`connect 192.168.254.253 8991`
login <username>	`login alice`
who	`who`
broadcast <message>	`broadcast Hello all!`
private <username> <message>	`private bob Meet after class.`
quit	`quit`

Responses from the server **must all have the following format**:

```
Status code
<EMPTY LINE>
Data section if required
```

Further explanation on the commands is found below:

1. The server will be started via command line with a user provided TCP port. This will be called the CONTROL PORT. For example:
    i. `python server.py 8991`
2. The server will use TCP to create a listening socket using the user provided port. You may use 8991 for testing as shown, however, the port provided at the command line **must** be used at runtime.
3. The server will listen for the following **commands** (in bold) from the client and:
    a) **connect** – The server will establish the connection with the client and respond with a different port number, the DATA PORT, for the client to receive data. The server creates a new socket for this data connection.
    Expected behavior:
        i. All commands will be received on the CONTROL PORT.
        ii. The response to the connect command will be sent to the client port provided in the connect connection handshake.
        iii. Responses to other commands will be sent on the DATA PORT.
    Example: the server may respond stating successful connection and to use data port 9000:

    ```
    200

    9000
    ```

    b) **login** – The server registers a user with the server and keeps track of the user appropriately to send future messages to.
    Expected behavior:
        i. Server validates uniqueness of username. Sends failure and stops processing request if name not unique.
        ii. User is added to active client list.
        iii. Server broadcasts user join notification.

    Example: the server broadcasts:

    ```
    200

    join
    alice
    ```

c) **who** – The server sends a command separated list of all usernames currently connected.
   Expected behavior:
   i. Server responds with a command separated list of all current users.
d) **broadcast** – The server sends a message to all connected users.
   Expected Behavior:
   i. Server forwards message to all active clients.
   ii. Sender name must appear with message.
   Example Format:

```
200

Broadcast
alice
Hello all!
```

e) **private** – The server sends a private message to one user. Expected Behavior:
   i. Message delivered only to specified recipient.
   ii. Error returned if recipient does not exist.
   iii. Example Recipient Format response:

```
200

Private
alice
Hello all!
```

   iv. Example Sender Format Response:

```
200
```

f) **quit** – Disconnects user from server.
   Expected Behavior:
   i. Server removes client from active list.
   ii. Control and Data sockets are closed.
   iii. Connections close gracefully.
   iv. Logout notification sent to remaining clients.

The Client

The following table outlines the commands the client will send. All commands will return **200** when the server executes the command successfully, and **500** when the command fails.

Command	Example
connect <ip> <port>	`connect 192.168.254.253 8991`
login <username>	`login alice`
who	`who`
broadcast <message>	`broadcast Hello all!`
private <username> <message>	`private bob Meet after class.`
quit	`quit`

Further explanation on the commands is found below:

1. The client will be started via command line. For example:
    i. `python client.py`
2. The user will then type the command to connect to the server using its IP address and the CONTROL PORT. For example:
    i. `connect 192.168.254.253 8991`

    The client will listen for a response from the server. Example: the server may respond and stating successful connection and to use data port 9000:

    ```
    200

    9000
    ```

3. Upon successfully connecting to the server, the following commands are sent over the CONTROL PORT with responses on the DATA PORT:
    i. **login** – The client sends the username it wishes to be known as to the server. Just a status code is expected as the server response.
    ii. **who** – The client requests a command separated list of all usernames currently connected. Example server response:

    ```
    200

    bob, mallory
    ```

    iii. **broadcast** – The client sends a message to the server requesting it to broadcast the message to all connected users. Example server response:

    ```
    200

    Broadcast
    alice
    Hello all!
    ```

iv. **private** – The client sends a request to the server requesting it to deliver a message to **only** the user indicated in the request.
v. **quit** – The client requests to be disconnected from the server. On success, it closes the connections and releases the resources. Just a status code is expected as the server response.

Output

Explicitly use the following output guideline template. “>” at the start of a line indicates a stage requiring user input or intervention:

1. The Server

```
> python server.py 8991

Starting server…

Creating server socket

Awaiting connections…

Connection requested. Creating data socket

Login requested by: bob

Connection requested. Creating data socket

Login requested by: alice

Who requested. Sending users.

Broadcast requested by alice
Message: Hello all!

Private message from alice to bob

Quit requested by alice

Quit requested by bob
```

2. The Sending Client

```
> python client.py

Starting client…

> connect 192.168.254.253 8991

200 status code received. Starting data connection on port
<DATA PORT>

> login alice

200 status code received. Login successful

> who
200 status code received. Users currently connected: bob,
mallory

> broadcast Hello all!

200 status code received.
Broadcast message from alice: Hello

> private bob Let’s talk

200 status code received. Message sent.

> quit

200 status code received.
```

3. The Receiving Client

```
> python client.py

Starting client…

> connect 192.168.254.253 8991

200 status code received. Starting data connection on port
<DATA PORT>

> login bob

200 status code received. Login successful

200 status code received.
Broadcast message from alice: Hello

200 status code received.
alice: Let’s talk

> quit

200 status code received.
```

Submission

1. A readme.txt file with the Group Member Names and IDS.
2. A copy of your final code to the LMS. (**Source code only (.py files). No zips. No IDE project files**). Do **not** submit zip files to file share site or links to zips at file share sites (including Google Drive). **Zero will be awarded without chance for review.**
3. A video recording (screen recording) of the execution of your team's project. The recording must show your application being executed for each graded part of the project. Narrate the video stating what is being shown.
4. Use Wireshark to show the messages being sent between the client(s) and the server. Ensure to show and highlight the message formats stated in the requirements. Wireshark will be a portion of the video grade.

You may upload the video to YouTube or Google Drive and submit the link to the LMS. **The link must be viewable at time of grading. Zero with no review will be awarded if it is not viewable at time of grading.**

Grading

The table below shows how the project will be graded. **Grading shown in the LMS and instructions from your instructor override all grading shown below**.

Item	**Percentage**
Server | connect	6
Server | login command	6
Server | who command	6
Server | broadcast	6
Server | private	6
Server | quit	5
Client | connect command	6
Client | login command	6
Client | who command	6
Client | broadcast	6
Client | private	6
Client | quit	5
video	30

Code plagiarism will be reported for academic dishonesty.